University of Balamand
Faculty of Arts & Sciences
CSIS 231 – Assignment 1: Project Proposal

PROJECT PROPOSAL

Sports Tournament Management System

A Client-Server Desktop Application for Tournament, Match, and Venue Management

Prepared by: Antonio Maroun	**Student ID:** A2412995
Prepared for: CSIS 231 – University of Balamand	**Date:** September 23, 2026

1. Idea & Expected Outcome

Problem Statement

Target users. The system has three types of users: **Team Managers** (who run a team's roster and enter it into tournaments), **Tournament Organizers** (who create tournaments, schedule matches, and enter results), and **Administrators** (who manage accounts and oversee the whole system).

The problem. Local sports tournaments are still coordinated through spreadsheets, WhatsApp groups, and phone calls. Organizers lose track of which teams are actually registered, Team Managers have no single place to manage a roster, and deciding where a team's matches will be played is handled informally — with no approval trail. This regularly causes double-booked courts, last-minute location changes, and matches scheduled at venues nobody confirmed were available.

Proposed Solution

The Sports Tournament Management System is a client-server desktop application: a JavaFX client talks to a Spring Boot server over JSON, and the server persists everything in PostgreSQL. Organizers create tournaments; Team Managers build a roster and register their team for a tournament. The feature at the center of this proposal is team venue registration: once a team is part of a tournament, its manager also registers, or proposes, a venue where that team wants to play its matches. A newly registered venue starts in PENDING status. An Organizer or Administrator reviews it and changes its status to APPROVED, REJECTED, or DISABLED — only an APPROVED venue can ever be assigned to an official match. Once a venue is approved, the Organizer schedules matches against it, the server checks for time conflicts, results are recorded after each match, and tournament standings recalculate automatically.

Security. Because Organizer and Administrator accounts can approve venues and change tournament data, both roles are protected by two-factor authentication (TOTP) in addition to a password, using the same standard as apps like Google Authenticator.

Success Criteria

- A Team Manager can create a team, add players, register it for a tournament, and propose a venue for that team's matches, all from the JavaFX client.
- An Organizer or Administrator can review a proposed venue and set its status to APPROVED, REJECTED, or DISABLED, and the server never lets a non-approved venue be attached to a scheduled match.
- The server refuses to double-book a venue, and refuses to schedule a team for two matches at overlapping times.
- Entering a match result automatically updates that tournament's standings — no manual recalculation.
- Organizer and Administrator logins require a valid TOTP code in addition to a password.

Scope

In scope: account authentication and roles with two-factor authentication for Organizers and Administrators; team and player management; tournament creation and team registration; team venue registration with an approval workflow (PENDING / APPROVED / REJECTED / DISABLED); venue availability and conflict checking; match scheduling; match results; and automatically updated tournament standings.

Out of scope this semester: online payments; live streaming; AI-based match predictions; GPS tracking; real-time chat; advanced analytics dashboards; a mobile client.

2. Data Model & ERD

The database has nine tables. The table below is the data dictionary: it lists every field with its type, and marks primary keys (PK), foreign keys (FK), unique constraints (UQ), NOT NULL (NN), CHECK constraints, and default values. Figure 1 shows the same structure as an ERD, with the cardinality of every relationship.

Table	Purpose	Fields (name, type, and constraints)
users	Every person who uses the system.	user_id SERIAL PK; name VARCHAR(100) NN; email VARCHAR(255) UQ NN; password_hash VARCHAR(255) NN; role VARCHAR(20) NN CHECK IN ('admin','organizer','team_manager'); two_factor_secret VARCHAR(64) NULL;

Table	Purpose	Fields (name, type, and constraints)
		two_factor_enabled BOOLEAN NN DEFAULT false; created_at TIMESTAMP NN DEFAULT now().
teams	A team managed by one Team Manager.	team_id SERIAL PK; team_name VARCHAR(100) NN; manager_id INT FK → users NN; created_at TIMESTAMP NN DEFAULT now().
players	A player on a team's roster.	player_id SERIAL PK; team_id INT FK → teams NN; first_name VARCHAR(50) NN; last_name VARCHAR(50) NN; jersey_number INT NN; position VARCHAR(30) NULL. UNIQUE (team_id, jersey_number).
tournaments	A tournament created by an Organizer.	tournament_id SERIAL PK; name VARCHAR(120) NN; organizer_id INT FK → users NN; start_date DATE NN; end_date DATE NN CHECK end_date >= start_date; status VARCHAR(20) NN DEFAULT 'draft' CHECK IN ('draft','open','in_progress','completed'); created_at TIMESTAMP NN DEFAULT now().
tournament_teams	Links a team to a tournament it has registered for (solves the many-to-many relation).	tournament_team_id SERIAL PK; tournament_id INT FK → tournaments NN; team_id INT FK → teams NN; status VARCHAR(20) NN DEFAULT 'pending' CHECK IN ('pending','approved','rejected'); registered_at TIMESTAMP NN DEFAULT now(). UNIQUE (tournament_id, team_id) — a team cannot register twice for the same tournament.
venues	A venue proposed by a team; when it may actually be used is stored separately in venue_availability.	venue_id SERIAL PK; venue_name VARCHAR(100) NN; location VARCHAR(150) NN; capacity INT NN CHECK > 0; surface_type VARCHAR(20) NN CHECK IN ('indoor','outdoor'); team_id INT FK → teams NN; status VARCHAR(20) NN DEFAULT 'pending' CHECK IN ('pending','approved','rejected','disabled'); created_at TIMESTAMP NN DEFAULT now().
venue_availability	One time window during which a venue can be booked; a venue can have several.	availability_id SERIAL PK; venue_id INT FK → venues NN; available_date DATE NN; start_time TIME NN; end_time TIME NN CHECK end_time > start_time.
matches	A scheduled match between two teams at a venue.	match_id SERIAL PK; tournament_id INT FK → tournaments NN; home_team_id INT FK → teams NN; away_team_id INT FK → teams NN CHECK home_team_id <> away_team_id; venue_id INT FK → venues NN; match_date DATE NN; start_time TIME NN; end_time TIME NN CHECK end_time > start_time; home_score INT NULL; away_score INT NULL; status VARCHAR(20) NN DEFAULT 'scheduled' CHECK IN ('scheduled','completed','cancelled').
standings	One team's aggregated record within one tournament.	standing_id SERIAL PK; tournament_id INT FK → tournaments NN; team_id INT FK → teams NN; played INT NN DEFAULT 0; won INT NN DEFAULT 0; drawn INT NN DEFAULT 0; lost INT NN DEFAULT 0; points INT NN DEFAULT 0; goal_diff INT NN DEFAULT 0. UNIQUE (tournament_id, team_id).

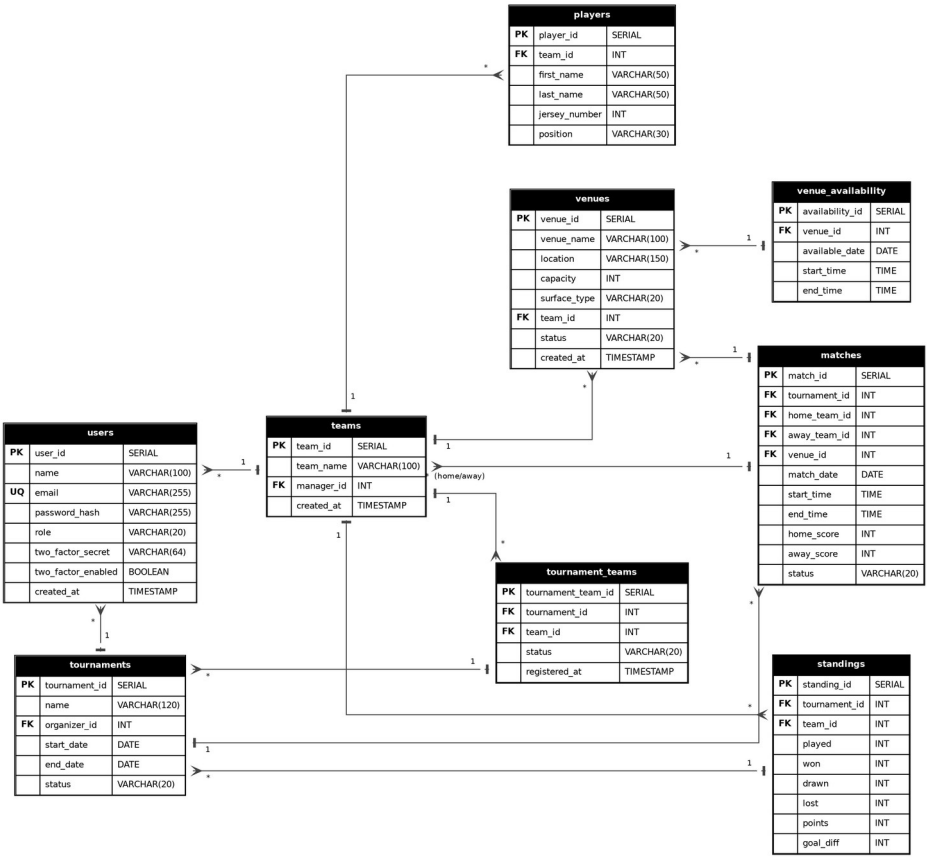


Figure 1. Entity-Relationship Diagram (ERD) – nine tables, Sports Tournament Management System.

Normalization Rationale

Most of the operational tables follow Third Normal Form: every fact is stored in exactly one place. A team's roster lives in players, not as a list inside teams, and a match references venue_id instead of repeating the venue's name and location on every row. The many-to-many relationship between teams and tournaments is split into the tournament_teams junction table, so a registration can have its own status (pending/approved/rejected) without duplicating team or tournament data. The same idea applies to matches, which store home_team_id and away_team_id as two separate foreign keys rather than two free-text team names.

One table is intentionally not fully normalized: standings stores a running total (played, won, points, …) per team per tournament, even though these numbers could be recalculated from matches on demand. It is kept as a maintained summary table instead, mainly so the Tournament Standings View can load instantly rather than recomputing a whole table from scratch every time someone opens it. Its values are simply updated by the service layer whenever a match result is entered or corrected, so this is a deliberate, documented exception rather than an inconsistency in the design.

The venue_availability table exists for a normalization reason too. An earlier version of this design stored a single available_date/available_from/available_to on the venues table itself, which meant a venue could only ever offer one time slot — a repeating group hidden inside a single row. Moving that information into its own table, with a foreign key back to venues, lets one venue offer several different days and time ranges, and keeps each one as a normal, atomic row instead of forcing the venues table to hold a variable number of time slots.

3. Service Layer

The server follows a standard Spring Boot layering: @RestController classes read the request and call a @Service; each service holds the business rules and calls a Spring Data JPA repository, which is the only layer that touches the database. Requests and responses are plain JSON DTOs (never JPA entities directly), validated with Bean Validation (@Valid) for basic checks such as required fields, plus explicit checks in the service layer for the business rules below.

Business Rules

- A team cannot register twice for the same tournament (UNIQUE constraint on tournament_teams).
- Two players on the same team cannot share the same jersey number (UNIQUE(team_id, jersey_number)); this does not by itself stop the same person being added twice under a different number, which is left to the Team Manager to avoid.
- A venue must have a non-empty name and location and a capacity greater than zero, and can only be registered or edited by the Team Manager who manages the owning team.
- A venue must be APPROVED before it can be scheduled; REJECTED or DISABLED venues can never be assigned to a match.
- Each availability period on a venue must have a start time before its end time, and a venue may have more than one such period.
- A match can only be scheduled if its date and time fall entirely inside one of the venue's availability periods.
- No double-booking: two matches cannot share a venue during overlapping times, and a team cannot be scheduled for two overlapping matches.
- Home and away teams on a match must be different, and both must already be APPROVED in tournament_teams for that tournament.
- A match result can be entered only while the match is still 'scheduled'; once entered, the match becomes 'completed' and any further change must go through updateMatchResult() instead.
- Whenever a result is entered or corrected, standings are recalculated for both teams straight away.
- A 2FA code must be exactly 6 digits and valid within the current 30-second TOTP window.

Selected Services in Detail

login() and verify2FA()

Purpose: Authenticate a user. Team Manager accounts may use a password alone; Organizer and Administrator accounts also need a TOTP code as a second step.

I/O: login(): email, password. verify2FA(): tempToken, code. → login(): a JWT if 2FA is off, otherwise a tempToken. verify2FA(): the JWT and user.

Tables/CRUD: users (Read) **Validation:** Credentials must match the stored hash; if two_factor_enabled is true, login() withholds the JWT and asks for verify2FA(); that call checks the 6-digit code against two_factor_secret within the current time window.

Status: 200 OK, 400 BAD REQUEST, 401 UNAUTHORIZED

registerVenue() and addVenueAvailability()

Purpose: A Team Manager proposes a venue for their team, then adds one or more time windows during which it can be booked.

I/O: registerVenue(): teamId, venueName, location, capacity, surfaceType. addVenueAvailability(): venueId, availableDate, startTime, endTime. → The created venue (status PENDING), and the created availability row.

Tables/CRUD: venues, venue_availability (Create) **Validation:** Team must exist and be managed by the caller; required fields cannot be empty; capacity > 0; startTime before endTime; a venue may have more than one availability row.

Status: 201 CREATED, 400 BAD REQUEST, 401 UNAUTHORIZED, 403 FORBIDDEN

checkVenueAvailability() and scheduleMatch()

Purpose: Before a match is created, the server checks the requested time against the venue's availability windows and existing matches; scheduleMatch() then creates the match if every check passes.

I/O: tournamentId, homeTeamId, awayTeamId, venueId, matchDate, startTime, endTime. → checkVenueAvailability(): true/false, plus any conflicting match. scheduleMatch(): the created match, status SCHEDULED.

Tables/CRUD: venues, venue_availability, matches (Read / Create) **Validation:** Venue must be APPROVED and the time must fall inside one of its availability rows; no other non-cancelled match at that venue may overlap;

both teams must be APPROVED in tournament_teams; homeTeamId ≠ awayTeamId; neither team may have an overlapping match.

Status: 200/201, 400 BAD REQUEST, 401 UNAUTHORIZED, 403 FORBIDDEN, 404 NOT FOUND, 409 CONFLICT — e.g. “Venue is already reserved during this time.”

enterMatchResult() and updateMatchResult()

Purpose: Record a match's final score once it has been played, and let an Organizer/Administrator correct that score later if needed.

I/O: matchId, homeScore, awayScore. → The match, now status COMPLETED, with standings recalculated for both teams.

Tables/CRUD: matches, standings (Update) **Validation:** enterMatchResult() only succeeds while the match is still 'scheduled'; updateMatchResult() only succeeds once it is already 'completed'. Either way the caller must be the tournament's Organizer or an Administrator, and calculateStandings() runs again for both teams.

Status: 200 OK, 400 BAD REQUEST, 401 UNAUTHORIZED, 403 FORBIDDEN, 404 NOT FOUND, 409 CONFLICT

createTeam(), addPlayer(), createTournament(), registerTeamInTournament()/approveTeamRegistration(), and the plain read/list/update/delete services follow this same pattern and are not repeated here — see the summary table below for their purpose, CRUD mapping, tables, and status codes.

Full Service Summary

Service	Purpose	CRUD	Tables affected	Status codes
login()	Authenticate with email + password	Read	users	200, 400, 401
verify2FA()	Second login step: checks the TOTP code, issues JWT	Read	users	200, 400, 401
setup2FA() / disable2FA()	Generate/clear the 2FA secret for the user	Update	users	200, 400, 401
createTeam()	Team Manager creates a new team	Create	teams	201, 400, 401
updateTeam()	Edit a team's own details	Update	teams	200, 400, 401, 403, 404
deleteTeam()	Remove a team with no tournament history	Delete	teams	204, 401, 403, 404, 409
getTeamById() / getAllTeams()	Fetch one team, or list teams	Read	teams	200, 404
addPlayer()	Add a player to a team's roster	Create	players	201, 400, 401, 403, 409
updatePlayer() / removePlayer()	Edit or remove a roster entry	Update / Delete	players	200/204, 401, 403, 404
getPlayersByTeam()	List a team's roster	Read	players	200, 404
createTournament()	Organizer creates a tournament	Create	tournaments	201, 400, 401, 403
updateTournament() / deleteTournament()	Edit/remove the organizer's own tournament	Update / Delete	tournaments	200/204, 401, 403, 404, 409
getAllTournaments() / getTournamentById()	Browse tournaments or view one	Read	tournaments	200, 404
registerTeamInTournament()	Team registers for a tournament	Create	tournament_teams	201, 400, 401, 403, 409
approveTeamRegistration()	Organizer approves a pending registration	Update	tournament_teams	200, 401, 403, 404, 409
removeTeamFromTournament()	Withdraw a team from a tournament	Delete	tournament_teams	204, 401, 403, 404
getTournamentTeams()	List teams registered in a tournament	Read	tournament_teams	200, 404
registerVenue()	Team Manager proposes a venue for the team	Create	venues	201, 400, 401, 403
updateVenue() / deleteVenue()	Edit or remove one of the team's own venues	Update / Delete	venues	200/204, 401, 403, 404, 409
getVenueById() / getVenuesByTeam() / getAllVenues()	Fetch one venue, a team's, or all venues	Read	venues	200, 404
approveVenue() / rejectVenue() / disableVenue()	Organizer/Admin changes a venue's status	Update	venues	200, 401, 403, 404, 409
addVenueAvailability() / updateVenueAvailability() / removeVenueAvailability()	Add, edit, or remove one availability window	Create/ Update/ Delete	venue_availability	200/201/204, 400, 401, 403, 404, 409
checkVenueAvailability()	Checks a time against availability & existing matches	Read	venue_availability, matches	200, 400, 404
scheduleMatch()	Organizer schedules a match at an approved, available venue	Create	matches	201, 400, 401, 403, 404, 409
updateMatch() / cancelMatch()	Reschedule or cancel a match	Update	matches	200, 401, 403, 404, 409
getMatchesByTournament() / getMatchesByTeam()	List matches for a tournament or a team	Read	matches	200, 404
enterMatchResult() / updateMatchResult()	Record or correct a match's result	Update	matches, standings	200, 400, 401, 403, 404, 409
calculateStandings() / getTournamentStandings()	Recompute (internal) or fetch a tournament table	Update / Read	standings	200, 404

Error Handling

A single @RestControllerAdvice class converts every exception into one JSON error shape ({ status, message, field? }), so the codes below are used the same way everywhere: 201 CREATED and 204 NO CONTENT for a successful create or delete; 400 BAD REQUEST for invalid input, such as a missing field or a bad TOTP format; 401 UNAUTHORIZED for a missing/invalid login, JWT, or TOTP code; 403 FORBIDDEN when the user is logged in but not allowed to do that action, such as a Team Manager trying to approve their own venue; 404 NOT FOUND when the team, tournament, venue, match, or player referenced does not exist; and 409 CONFLICT for anything that clashes with the current state of the data, such as a duplicate tournament registration, a venue that is already booked, or a team that is already scheduled during that time.

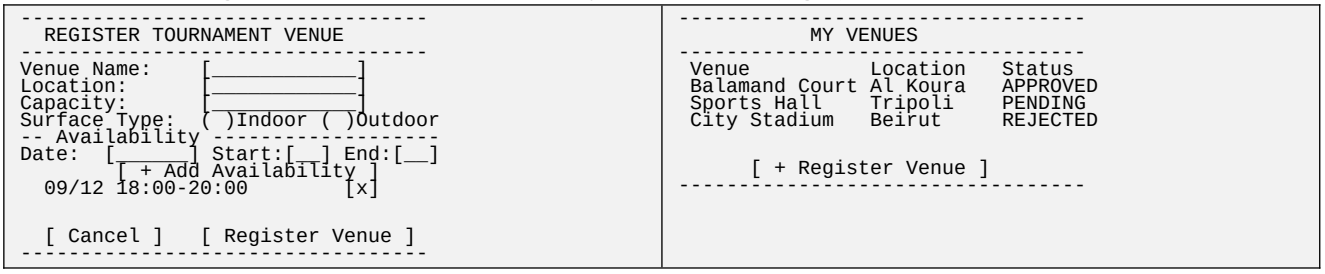
4. Client Views (JavaFX)

The client follows an MVC/MVVM split: each screen is an FXML view paired with a Controller class. A Session singleton holds the JWT and the logged-in user's role once login (and, if enabled, the 2FA step) succeeds, and every later call goes through a shared ApiClient that attaches the token and parses the JSON response into model classes. A SceneManager swaps the root of the primary Stage to move between views, and each Controller reads

Session.getRole() to hide actions the current role should not see — for example, only an Organizer or Administrator ever sees the Approve/Reject/Disable buttons on the Venue Approval View. The server checks every rule again regardless of what the client hides, since the client-side check only makes the interface friendlier.

View	Purpose & key info	Services called	Navigation
Login View	Email + password, then a 2FA code field when the account has 2FA on.	login(), verify2FA()	→ Dashboard on success
Security Settings	Turn 2FA on/off; shows QR code and backup codes.	setup2FA(), disable2FA()	→ from Dashboard menu
Dashboard	Landing screen after login; shortcuts based on role.	getAllTournaments()	→ Teams, Tournaments, My Venues
Teams View	List the Team Manager's teams.	getAllTeams()	→ Team Details
Team Details View	One team's info and roster summary.	getTeamById(), getPlayersByTeam()	→ Players, edit team
Players View	Add/edit/remove players on a team.	addPlayer(), updatePlayer(), removePlayer()	→ back to Team Details
Tournaments View	Browse all tournaments.	getAllTournaments()	→ Tournament Details
Tournament Details View	One tournament's info, registered teams, standings link.	getTournamentById(), getTournamentTeams()	→ Registration, Standings, Match Schedule
Tournament Registration View	Register one of the manager's teams for this tournament.	registerTeamInTournament()	→ back to Tournament Details
My Venues View	The manager's own venues and their status.	getVenuesByTeam()	→ Register/Edit/Details Venue
Register Venue View	Form to propose a venue, with an Availability section to add date/time windows.	registerVenue(), addVenueAvailability()	→ back to My Venues
Edit Venue View	Edit a PENDING venue, including its availability windows.	getVenueById(), updateVenue(), updateVenueAvailability(), removeVenueAvailability()	→ back to My Venues
Venue Details View	Full details of one venue.	getVenueById()	→ My Venues / Venue Approval
Venue Approval View	Organizer/Admin reviews a PENDING venue.	getVenueById(), approveVenue(), rejectVenue(), disableVenue()	→ back to venue queue
Match Schedule View	Calendar/list of a tournament's matches.	getMatchesByTournament(), checkVenueAvailability(), scheduleMatch()	→ Match Details
Match Details View	One match's teams, venue, and time.	getMatchesByTeam()	→ Enter Match Result
Enter Match Result View	Form to record the final score, or correct one already entered.	enterMatchResult(), updateMatchResult()	→ Tournament Standings
Tournament Standings View	Points table for a tournament.	getTournamentStandings()	—

Wireframes — Register Venue View (left) and My Venues View (right)



Error Handling

Field-level problems (a blank required field, a non-numeric capacity) are validated in the Controller before the call is sent, and shown inline under the field in red. A JSON error returned by the server — for example the 409 Conflict from scheduleMatch() when a venue is already booked — is shown as a banner at the top of the view, using the message field from the central error handler.

5. Risks & Mitigations

Risk	Mitigation
A venue or a team gets double-booked for overlapping matches.	checkVenueAvailability() is checked as part of the same operation that creates the match in scheduleMatch(), so the two steps cannot be separated by another request sneaking in between them.
A team is scheduled before its venue is approved.	scheduleMatch() rejects any venue whose status is not APPROVED, both in the service layer and with a database constraint.
The project grows beyond what is realistic for one semester.	The in/out-of-scope list in Section 1 is fixed for the semester; features such as payments, streaming, and analytics stay explicitly out.
Connecting the JavaFX client to the Spring Boot server takes longer than expected.	During the first weeks, the login screen and the Teams view will be built and connected end-to-end before any other feature, so integration problems between the client and the server show up early instead of near the deadline.
An Organizer or Administrator loses access to their 2FA device.	One-time backup codes are issued when 2FA is set up, and an Administrator can also reset 2FA on an account after confirming the person's identity.

6. Milestones

The schedule below is written as week numbers so it can be lined up with the actual semester calendar once classes start.

Weeks	Focus	Key deliverables
Weeks 1–3	Setup	Final ERD, SQL schema with constraints, Maven/Git project skeleton for the client and server, and a working JavaFX ↔ Spring Boot connection through a first login screen.
Weeks 4–6	Auth & core entities	JWT authentication, TOTP/2FA for Organizer and Administrator accounts, TeamService and PlayerService, and the Login, Security Settings, Dashboard, Teams, and Players views.
Weeks 7–9	Tournaments	TournamentService, tournament registration (tournament_teams), and the Tournaments, Tournament Details, and Tournament Registration views.
Weeks 10–12	Venues	VenueService and venue_availability, the full approval workflow, and the Register Venue, My Venues, and Venue Approval views.
Weeks 13–14	Matches & standings	MatchService with conflict checking, enterMatchResult()/updateMatchResult(), StandingsService, and the remaining match/standings views.
Week 15	Testing & delivery	Tests on the conflict-checking rules, UI polish, documentation, and the final demo.